

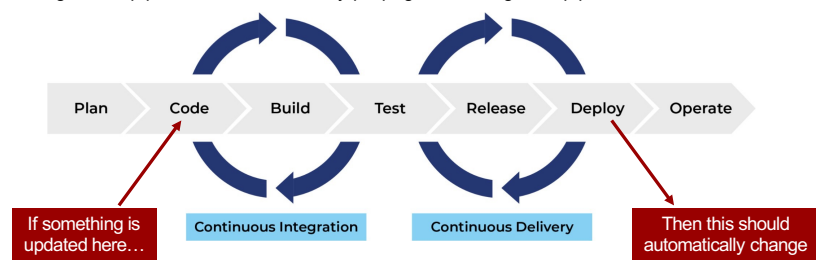
Continuous Integration

ONE LOVE. ONE FUTURE.

1

Continues X

🔥 Term refers to a set of software practices for automating tedious tasks and make sure changed in a pipeline are continuously propagated through the pipeline



2

CI	CD	CML
Continues Integration	Continues Deployment	Continues Machine Learning
Core tasks: 💡 How to automatically secure that code does not break during development?	Core tasks: 💡 How to get your code/application to the user automatically? + monitor life cycle	Core tasks: 💡 How to automatically retrain machine learning models when data and code changes?
🖥️ App independent concept	🖥️ App dependent concept	🖥️ Specific to ML applications




ĐẠI HỌC BÁCH KHOA HÀ NỘI
 HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

3

MLOps levels

The **Maturity model** overall describes the DevOps practices to run a successful MLOps environment.

Intended to identify gaps in an existing organization's attempt to implement such an environment.

💡 Estimate the scope of the work for new engagements.

💡 Establish realistic success criteria.

💡 Identify deliverables you'll hand over at the conclusion of the engagement.

Level	Description	Highlights	Technology
0	No MLOps	- Difficult to manage full machine learning model lifecycle - The teams are disparate and releases are painful - Most systems exist as "black boxes," little feedback during/post deployment	- Manual builds and deployments - Manual testing of model and application - No centralized tracking of model performance - Training of model is manual
1	DevOps but no MLOps	- Releases are less painful than No MLOps, but rely on Data Team for every new model - Still limited feedback on how well a model performs in production - Difficult to trace/reproduce results	- Automated builds - Automated tests for application code
2	Automated Training	- Training environment is fully managed and scalable - Easy to reproduce model - Releases are manual, but low friction	- Automated model training - Centralized tracking of model training performance - Model management
3	Automated Model Deployment	- Releases are low friction and automatic - Full traceability from deployment back to original data - Entire environment managed: train > test > production	- Integrated A/B testing of model performance for deployment - Automated tests for all code - Centralized tracking of model training performance
4	Full MLOps Automated Operations	- Full system automated and easily monitored - Production systems are providing information on how to improve and, in some cases, automatically improve with new models - Approaching a zero-downtime system	- Automated model training and testing - Verbose, centralized metrics from deployed model

ĐẠI HỌC BÁCH KHOA HÀ NỘI
 HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

4

Continues integration

Core task:

🔥 How to automatically secure that code does not break during development? 🔥

3 steps to do this:

💡 Use version control

Frequently committing code to a shared repository

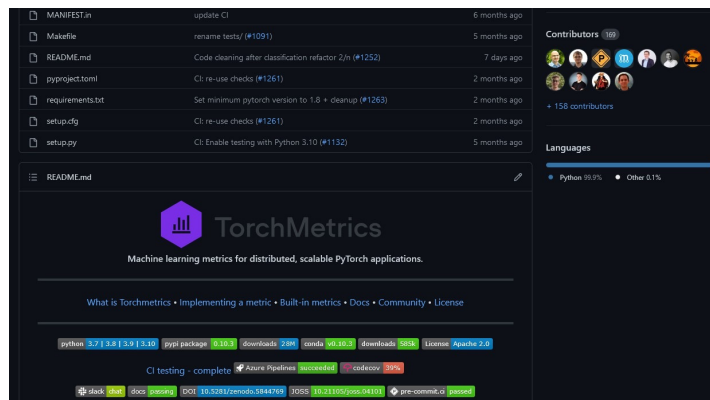
💡 Write (unit)test for your code

Should capture unwanted bugs in your code

💡 Automate build + testing

Automatically run test so code cannot be merged without working

A small case study for continuous integration



The screenshot shows the GitHub repository for TorchMetrics. The commit history table is as follows:

File	Commit Message	Time Ago
MANIFEST.in	update CI	6 months ago
Makefile	rename tests/ (#1091)	5 months ago
README.md	Code cleaning after classification refactor 2/n (#1252)	7 days ago
pyproject.toml	CI: re-use checks (#1261)	2 months ago
requirements.txt	Set minimum pytorch version to 1.8 + cleanup (#1263)	2 months ago
setup.cfg	CI: re-use checks (#1261)	2 months ago
setup.py	CI: Enable testing with Python 3.10 (#1132)	5 months ago

The README.md section shows the TorchMetrics logo and the text: "Machine learning metrics for distributed, scalable PyTorch applications." It also includes links for "What is Torchmetrics", "Implementing a metric", "Built-in metrics", "Docs", "Community", and "License".

At the bottom, there are badges for CI testing status: "CI testing - complete", "Azure Pipelines - succeeded", "codecov - 100%", "slack - chat", "docs - preview", "DOI - 10.26434/chemrxiv-2020-08447", "JOSS - 10.26434/chemrxiv-2020-08447", and "pre-commit.ci - passed".

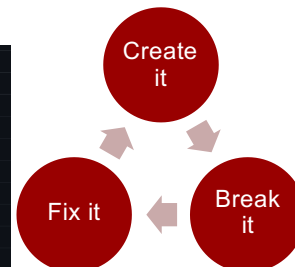
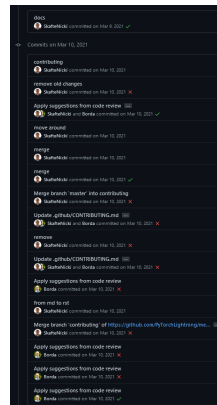
CI step 1: version control

User version control:

- 💡 Code changes are tracked
- 💡 Branches for parallel work

Commit frequently:

- 💡 Catch errors sooner than later
- 💡 Revert back easily to when things were working
- 💡 Merging can be done automatically



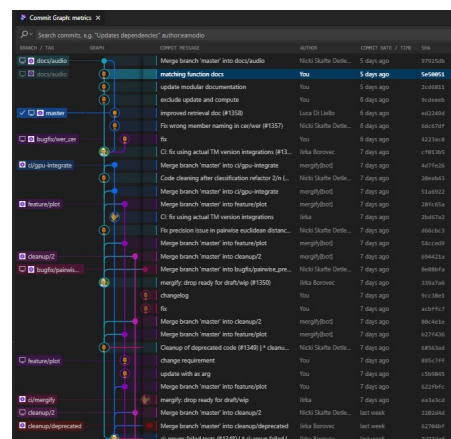
CI step 1: Use branches

Parallel workflow

Experimental features changes are kept away from master/main

Recommend extensions for VS code:

- 💡 [Gitlens](#) or [GitGraph](#)
- 💡 [Github PR and issues](#)



CI step 1: Use pull requests

- ▲ No commit can be pushed to master without being in a pull request

The screenshot shows a GitHub pull request (PR) for the repository 'Lightning-AI / metrics'. The PR is titled 'Added MinkowskiDistance support #1362'. The interface is divided into several sections:

- 1. Find PR:** The PR is listed in the 'Pull requests' tab.
- 2. Check changes:** The 'Files changed' tab shows the changes made in the PR, including files like 'functional_regression_map.py'.
- 3. Make one or more comments:** A comment box is visible at the bottom of the PR, with a 'Reply' button.
- 4. Send review:** A 'Review' button is visible at the bottom right of the PR.

The bottom of the screenshot shows the 'HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY' logo.

9

CI step 1: pre-commit



- ✓ Check that everything is up to standard before commits are created

The screenshot shows a terminal window with the output of a pre-commit hook. The output is as follows:

```

git commit -m "Implementation of client"
fix end of files.....Failed
- hook id: end-of-file-fixer
- exit code: 1
- files were modified by this hook

Fixing s8_monitoring/exercise_files/client.py

trim trailing whitespace.....Passed
check toml.....(no files to check)Skipped
check docstring is first.....Passed
check that executables have shebangs.....Passed
check for case conflicts.....Passed
detect private key.....Passed
ruff.....Failed
- hook id: ruff
- exit code: 1
- files were modified by this hook

s8_monitoring/exercise_files/client.py:17:12: S113 Probable use of requests call without timeout
Found 2 errors (1 fixed, 1 remaining).

ruff-format.....Passed
codespell.....Passed
markdownlint-docker.....(no files to check)Skipped
  
```

The bottom of the screenshot shows the 'HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY' logo.

10

CI step 2: write tests

Tests are the cornerstones of continuous integration

💡 *unit tests* are arguable the most important.

💡 A single unittest, tests a small part of your code

💡 By testing code in small pieces, bugs are easier to find

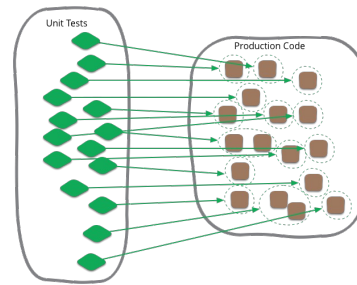
Other test types worth considering:

🔥 Integration tests

🔥 Regression tests

🔥 Performance tests

🔥 Security tests



CI step 2: write tests

💡 By Python convention your source code should either be

src/<project_name>
(src-layout)

or

<project_name>
(flat-layout)

File/Directory	Commit Message	Commit Hash	Time Ago
audio	CI re-use checks	#1281	2 months ago
classification	Code cleaning after classification refactor 2in	#1252	7 days ago
detection	Revert nndp matching reclassification	#1327	19 days ago
functional	Code cleaning after classification refactor 2in	#1252	7 days ago
image	Add normalize argument to certain image metrics	#1246	21 days ago
multimodal	Add CLIP score	#1314	11 days ago
nominal	Add Three's U-Statistic (Uncertainty) Metric	#1375	10 days ago
regression	Add LogCash Error	#1316	21 days ago
retrieval	CI re-use checks	#1281	2 months ago
text	Fix wrong member naming in reviewer	#1357	6 days ago
utils	Code cleaning after classification refactor 2in	#1252	7 days ago
wrappers	Code cleaning after classification refactor 2in	#1252	7 days ago
_about.py	docs: Update the number of metrics + covered domains	#1344	11 days ago
init.py	Fact match multiclass	#1343	8 days ago
aggregation.py	move torchmetrics >> src		6 months ago
collections.py	Code cleaning after classification refactor 2in	#1252	7 days ago
metric.py	Cleanup of deprecated code	#1348	7 days ago
pytyped	move torchmetrics >> src		6 months ago

CI step 2: write tests

- For tests, the convention is to either place the tests in a separate tests folder, or put the tests in the same folder as the function/class/submodule they are testing.

```

• | README.md
• | src/
• | | __init__.py
• | | important_functions.py
• | tests/
• | | __init__.py
• | | test_important_functions.py

```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

File	Commit Message	Author	Time
audio	Set minimum pytorch version to 1.8 + cleanup (#1263)	6 authors	2 months ago
bases	Code cleaning after classification refactor 2/n (#1252)	6 authors	7 days ago
classification	Cleanup of deprecated code (#1348)	6 authors	7 days ago
detection	Revert mAP matching vectorization (#1327)	6 authors	19 days ago
helpers	Add CLIP score (#1314)	6 authors	11 days ago
image	Add normalize argument to certain image metrics (#1246)	6 authors	21 days ago
multimodal	Add CLIP score (#1314)	6 authors	11 days ago
nominal	Add Thiel's U Statistic (Uncertainty) Metric (#1337)	6 authors	10 days ago
pairwise	Fix precision issue in pairwise euclidean distance (#1352)	6 authors	7 days ago
regression	Add LogCosh Error (#1316)	6 authors	21 days ago
retrieval	[Refactor] Classification 1/n (#1054)	6 authors	3 months ago
test	Bugfix: Update message regarding connection issues to the HF hub (#141)	6 authors	5 months ago
utilities	Set minimum pytorch version to 1.8 + cleanup (#1263)	6 authors	2 months ago
wrappers	Code cleaning after classification refactor 2/n (#1252)	6 authors	7 days ago
__init__.py	rename test/ (#1081)	6 authors	6 months ago

13

CI step 2: write tests

- In python, we recommend using the **pytest** framework.
- Test are simple functions that start with `test_` and uses `assert`

```

import torch
from torch.nn.functional import mse_loss

def test_mse_loss_zeros():
    # (0 - 0)**2 = 0
    assert mse_loss(torch.zeros(1,), torch.zeros(1,)) == 0

def test_mse_loss_ones():
    # (1 - 0)**2 = 1
    assert mse_loss(torch.ones(1,), torch.zeros(1,)) == 0

```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

14

CI step 2: write tests

Test can be simple...

```
def test_warning_on_nan(tmpdir):
    preds = torch.randint(3, size=(20,))
    target = torch.randint(3, size=(20,))

    with pytest.warns(
        UserWarning,
        match='.* nan values found in confusion matrix have been replaced with zeros.',
    ):
        confusion_matrix(preds, target, num_classes=5, normalize='true')
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

15

CI step 2: write tests

Test can be simple...

```
def test_warning_on_nan(tmpdir):
    preds = torch.randint(3, size=(20,))
    target = torch.randint(3, size=(20,))

    with pytest.warns(
        UserWarning,
        match='.* nan values found in confusion matrix have been replaced with zeros.',
    ):
        confusion_matrix(preds, target, num_classes=5, normalize='true')
```

Or complicated

```
@pytest.mark.parametrize("normalize", [True, 'pred', 'all', None])
@pytest.mark.parametrize(
    "preds, target, sk_metric, num_classes, multilabel",
    [
        (InputBinaryProb, InputBinaryTarget, sk_cm_binary_prob, 2, False),
        (InputBinaryLogits, InputBinaryLogitsTarget, sk_cm_binary_prob, 2, False),
        (InputBinaryProb, InputBinaryTarget, sk_cm_binary, 2, False),
        (InputMlbProb, InputMlbTarget, sk_cm_multilabel_prob, NUM_CLASSES, True),
        (InputMlbLogits, InputMlbLogitsTarget, sk_cm_multilabel_prob, NUM_CLASSES, True),
        (InputMlbProb, InputMlbTarget, sk_cm_multilabel, NUM_CLASSES, True),
        (InputMclsProb, InputMclsTarget, sk_cm_multiclass_prob, NUM_CLASSES, False),
        (InputMclsLogits, InputMclsLogitsTarget, sk_cm_multiclass_prob, NUM_CLASSES, False),
        (InputMclsProb, InputMclsTarget, sk_cm_multiclass, NUM_CLASSES, False),
        (InputMmcProb, InputMmcTarget, sk_cm_multidim_multiclass_prob, NUM_CLASSES, False),
        (InputMmcProb, InputMmcTarget, sk_cm_multidim_multiclass, NUM_CLASSES, False)
    ]
)

class TestConfusionMatrix(MetricTester):
    @pytest.mark.parametrize("ddp", [True, False])
    @pytest.mark.parametrize("dist_sync_on_step", [True, False])
    def test_confusion_matrix(
        self, normalize, preds, target, sk_metric, num_classes, multilabel, ddp, dist_sync_on_step
    ):
        self.run_class_metric_test(
            ddp=ddp,
            preds=preds,
            target=target,
            metric_class=ConfusionMatrix,
            sk_metric=partial(sk_metric, normalize=normalize),
            dist_sync_on_step=dist_sync_on_step,
            metric_args={
                "num_classes": num_classes,
                "threshold": THRESHOLD,
                "normalize": normalize,
                "multilabel": multilabel
            }
        )
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

16

CI step 2: execute locally

```
(lightning) C:\Users\vsde\Documents\metrics\pytest tests\unittests\regression\test_mean_error.py
===== test session starts =====
platform win32 -- Python 3.8.13, pytest-6.2.5, py-1.11.0, pluggy-1.0.0
rootdir: C:\Users\vsde\Documents\metrics, configfile: setup.cfg
plugins: cov-4.0.0, doctestplus-0.12.1, timeout-2.1.0
collected 116 items

tests\unittests\regression\test_mean_error.py .....XXXXXXXXX

----- warnings summary -----
..\.venv\Anaconda3\envs\lightning\lib\site-packages\pytest\config\_init_.py:1183
C:\Users\vsde\Anaconda3\envs\lightning\lib\site-packages\pytest\config\_init_.py:1183: PytestDeprecationWarning: The --strict option is deprecated, use --strict-markers instead.
  self.issue_config_time_warning(
-- Docs: https://docs.pytest.org/en/stable/warnings.html
===== 90 passed, 28 skipped, 8 xfailed, 1 warning in 16.44s =====
```

- ✓ Test passed
- ✗ Test failed
- S Test skipped (pytest.skipif, pytest.skip)
- X Test was expected to fail (pytest.xfail)

Do you remember to do this before each commit?
Let's automate doing it instead



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

17

CI step 3: Automating stuff

What can be automated: EVERYTHING 🤖

- 💡 Unit testing
- 💡 Integration testing
- 💡 Documentation creation
- 💡 Linters (style formatting)
- 💡 Security checks
- 💡 Code coverage
- 💡 Custom checks...

Only your imagination is the limit...



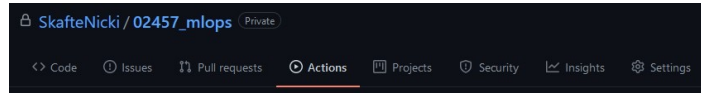
ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

18

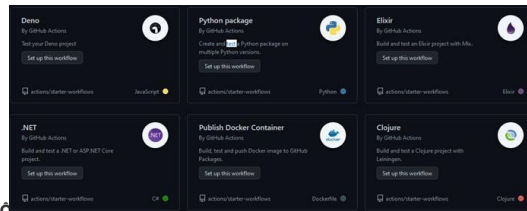
CI step 3: Github actions

Build-in continuous integration in Github.

Free 2000 automation minutes/month (public repository)



Many ready to go workflows



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

19

CI step 3: workflow files^{triggered}

Workflow files are a set of instructions that should be executed on a virtual machine hosted by Github

You can have one or many workflow files (runs in parallel)

When should workflow be

Define OS + python

Clones code

Setup Python

Install dependencies

Check formatting

Run tests

```

1 name: Python package
2
3 on:
4   push:
5     branches: [ main ]
6   pull_request:
7     branches: [ main ]
8
9 jobs:
10  build:
11
12    runs-on: ubuntu-latest
13
14    strategy:
15      matrix:
16        python-version: ["3.7", "3.8", "3.9", "3.10"]
17
18    steps:
19      - uses: actions/checkout@v3
20      - name: Set up Python ${{ matrix.python-version }}
21        uses: actions/setup-python@v4
22        with:
23          python-version: ${{ matrix.python-version }}
24      - name: Install dependencies
25        run: |
26          python -m pip install --upgrade pip
27          pip install flake8 pytest
28          pip install -r requirements.txt
29          python setup.py install
30      - name: Lint with flake8
31        run: |
32          flake8 src/
33      - name: Test with pytest
34        run: |
35          pytest tests/

```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

20

CI step 3: workflow files

```

- name: Install dependencies
  run: |
    python -m pip install --upgrade --user pip
    pip install --requirement ./requirements.txt --find-links https://download.pytorch.org/whl/cpu/torch_stable.html
    python --version
    pip --version
    pip list
  shell: bash

- name: Test Package [only]
  run: |
    # NOTE: run coverage on tests does not propagate faler status for Win, https://github.com/pytest-dev/pytest-cov/issues/1003
    python -m pytest torchmetrics -v --cov=torchmetrics --junitxml=junit/test-results-{{ runner.os }}-{{ matrix.python-version }}.xml
  
```

21

CI step 3: Workflow files

✓ 43 checks in total

Test a combination of

- 💡 Hardware setup
- 💡 Operating system
- 💡 Python version
- 💡 Core dependencies

Runs unit tests, build documentation, test coverage, linting of code, package installer etc.

✓ Summary	CI testing - complete on: pull_request	General checks on: pull_request
✓ WIP	✓ check-diff / eval-diff	✓ check-code / mpy
✓ Doc check on: pull_request	✗ pytest (ubuntu-20.04, 3.8, 1.8.1)	✗ check-code / pre-commit
✓ test-docs	✗ pytest (ubuntu-20.04, 3.8, 1.10.2)	✓ check-schema / schema
✓ make-docs	✗ pytest (ubuntu-20.04, 3.8, 1.11.0)	✓ check-package / pkg-check (ub...
✓ paper-JOSS	✗ pytest (ubuntu-20.04, 3.8, 1.12.1)	✓ check-package / pkg-check (ub...
✓ paper-cite	✗ pytest (ubuntu-22.04, 3.8, 1.13.0)	✓ check-package / pkg-check (ma...
✓ CI integrations on: pull_request	✗ pytest (ubuntu-22.04, 3.10, 1.13...	✓ check-package / pkg-check (ma...
✓ pytest (ubuntu-20.04, 3.7, oldest)	✗ pytest (ubuntu-22.04, 3.8, 1.13.0)	✓ check-package / pkg-check (wi...
✓ pytest (ubuntu-20.04, 3.10, latest)	✗ pytest (ubuntu-22.04, 3.9, 1.13.0)	✓ check-code / mpy
✓ pytest (macOS-11, 3.7, oldest)	✗ pytest (ubuntu-20.04, 3.7, 1.8.1...	✗ check-code / pre-commit
✓ pytest (macOS-11, 3.7, latest)	✗ pytest (ubuntu-20.04, 3.8, 1.8.1...	✓ check-schema / schema
✓ pytest (macOS-11, 3.10, latest)	✗ pytest (macOS-11, 3.8, 1.8.1, old...	✓ check-package / pkg-check (ub...
✓ pytest (windows-2022, 3.7, oldest)	✗ pytest (macOS-11, 3.7, 1.8.1, old...	✓ check-package / pkg-check (ma...
✓ pytest (windows-2022, 3.7, oldest)	✗ pytest (macOS-11, 3.9, 1.13.0)	✓ check-package / pkg-check (ma...
✓ pytest (windows-2022, 3.10, latest)	✗ pytest (windows-2022, 3.8, 1.13.0)	✓ check-package / pkg-check (wi...
✓ pytest (B.10, latest, ubuntu-22.04)	✗ pytest (windows-2022, 3.7, 1.8.1...	✓ check-code / mpy
✓ pytest (B.10, latest, macOS-11)	✗ pytest (windows-2019, 3.8, 1.8.1...	✗ check-code / pre-commit

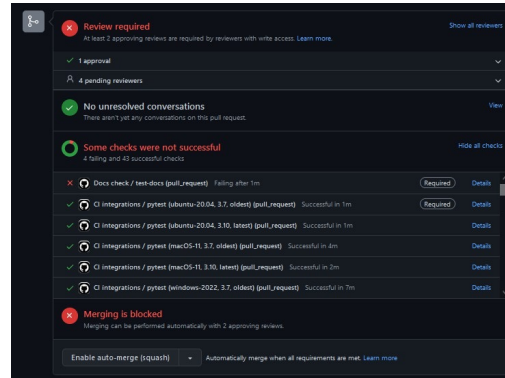
22

CI step 3: Code is checked before merging

Branch protection rules:

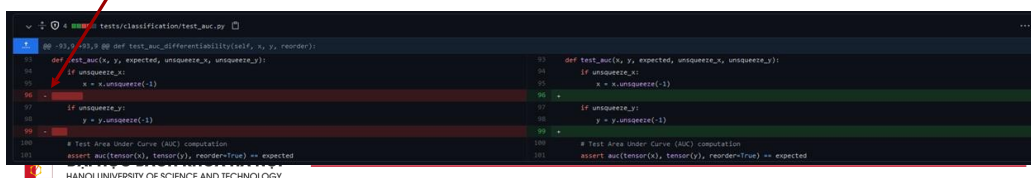
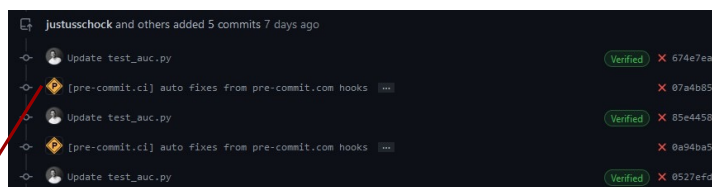
- ⚠️ Some tests should pass
- ⚠️ At least x core members need to approve
- ⚠️ Comments should be taken care of

View more [here](#)



23

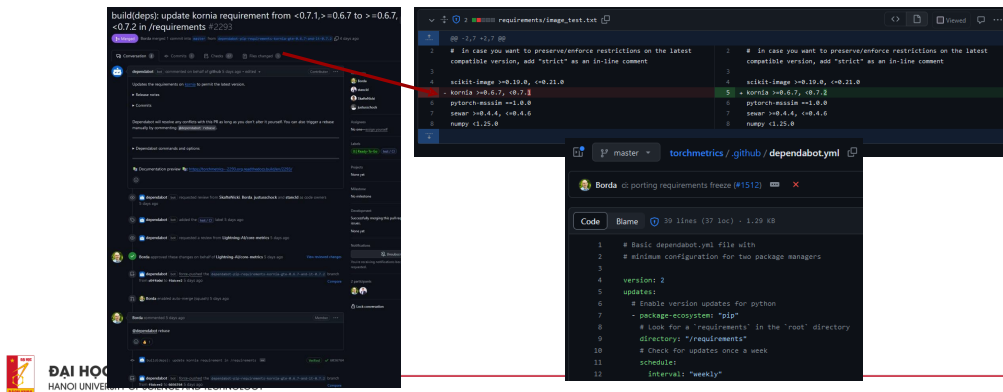
CI step 3: Automate tedious tasks with bots



24

CI step 3: Automate tedious tasks with bots

🤖 Dependabot can help auto checking new releases of dependencies in your project



The image shows three screenshots related to Dependabot:

- Left:** A GitHub repository page for `requirements/requirements.txt` showing a pull request from Dependabot to update `kornia` from `<0.7.1,>=0.6.7 to >=0.6.7,<0.7.2 in /requirements` (PR #2293).
- Top Right:** A code editor showing the `requirements/requirements.txt` file with the following content:


```
1 # In case you want to preserve/enforce restrictions on the latest
2 # compatible version, add "strict" as an in-line comment
3
4 kornia==0.6.7, <0.7.2
5
6 kornia==0.6.7, <0.7.2
7
8 kornia==0.6.7, <0.7.2
9
10 kornia==0.6.7, <0.7.2
11
12 kornia==0.6.7, <0.7.2
```
- Bottom Right:** A code editor showing the `dependabot.yml` file with the following content:


```
1 # Basic dependabot.yml file with
2 # minimum configuration for two package managers
3
4 version: 2
5 updates:
6   # Enable version updates for python
7   - package-ecosystem: "pip"
8     # Look for a "requirements" in the "root" directory
9     directory: "/requirements"
10    # Check for updates once a week
11    schedule:
12      interval: "weekly"
```

25

Summary of continues integration

1. Use version control
2. Write (unit-)test for your code
3. Automate build + test



26